

Lagrangian Neural Network-Based Control: Improving Robotic Trajectory Tracking via Linearized Feedback

Manuel Weiss¹, Alexander Pawluchin¹, Jan-Hendrik Ewerling¹, Thomas Seel¹, *Member, IEEE*, and Ivo Boblan

Abstract—This letter introduces a control framework that leverages Lagrangian neural network (LNN) for computed torque control (CTC) of robotic systems with unknown dynamics. Unlike prior LNN-based controllers that are placed outside the feedback-linearization framework (e.g., feedforward), we embed an LNN inverse-dynamics model within a CTC loop, thereby shaping the closed-loop error dynamics. This strategy, referred to as LNN-CTC, ensures a physically consistent model and improves extrapolation, requiring neither prior model knowledge nor extensive training data. The approach is experimentally validated on a robotic arm with four degrees of freedom and compared with conventional model-based CTC, physics-informed neural network (PINN)-CTC, deep neural network (DNN)-CTC, an LNN-based feedforward controller, and a PID controller. Results demonstrate that LNN-CTC significantly outperforms model-based baselines by up to 30 % in tracking accuracy, achieving high performance with minimal training data. In addition, LNN-CTC outperforms all other evaluated baselines in both tracking accuracy and data efficiency, attaining lower joint-space RMSE for the same training data. The findings highlight the potential of physics-informed neural architectures to generalize robustly across various operating conditions and contribute to narrowing the performance gap between learned and classical control strategies.

Index Terms—Machine learning for robot control, model learning for control, motion control.

I. INTRODUCTION

THE ever-increasing complexity of robotic systems demands advanced, data-efficient control methods to handle nonlinear dynamics and ensure precision in real-world applications such as healthcare, manufacturing, and autonomous navigation [1].

Linearized feedback control transforms nonlinear robot dynamics into linear systems, enabling systematic stability analysis

and precise trajectory tracking. This approach is particularly advantageous in applications that demand interpretability, robustness, and tunable performance. However, its effectiveness hinges on accurate dynamics models. For complex robots, such fidelity is often impractical, and the necessary modeling simplifications degrade regulation and tracking performance [2], [3]. CTC is a standard feedback-linearization method in robotics. It uses a robot model to cancel inertia, Coriolis/centrifugal, and gravity terms, thereby reducing the closed-loop error dynamics to a decoupled linear form. In response to these model-fidelity limitations, learning-based controllers fit policies or dynamics directly from data. Deep reinforcement learning (RL) often requires tens of millions of interactions to obtain effective locomotion policies. On hardware, this translates to extensive data collection [4]. Beyond RL-specific data demands, noisy supervision (e.g., imperfect rewards or demonstrations) further hurts sample efficiency, underscoring that data quality is as critical as data quantity [5]. More generally, DNN-based controllers tend to be black-box and hard to interpret and often generalize poorly outside of the training distribution. As a result, performance is typically reliable only within the regions represented in the training data [6], [7], [8].

Recent advances enable learning of robot dynamics with physics-informed models such as PINN [9], [10] and LNN [11], [12], embedding mechanics priors to improve data efficiency and generalization [9]. Their effectiveness has been shown in control, e.g., PINN for state prediction in model predictive control (MPC) [9] and accurate continuum robot modeling [10], while structured mechanical model (SMM) and LNN outperform black-box DNN under limited data across simulated robots [13]. LNN are specialized PINN tailored to systems governed by Lagrangian mechanics, enforcing physical consistency and enabling robust extrapolation from small datasets [11]. However, in prior work, LNN controllers are typically placed outside the feedback-linearization framework, used either as feedforward terms or within low-rate MPC. While feedforward LNN provide fast tracking, they offer limited disturbance rejection when the state deviates from the reference. The adaptive LNN-MPC runs at 50 Hz due to solver latency and model-update overhead and therefore relies on a secondary proportional derivative (PD) loop at 1 kHz for real-time stabilization [14]. As a result, this control strategy may not be suitable for highly dynamic systems.

To our knowledge, this work is the first to embed an inverse-dynamics LNN inside CTC, altering the closed-loop dynamics

Received 29 July 2025; accepted 22 November 2025. Date of publication 14 January 2026; date of current version 20 January 2026. This article was recommended for publication by Associate Editor H. Li and Editor C. Gosselin upon evaluation of the reviewers' comments. This work was supported by Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – FIP-12 under Project 528483508. (Corresponding author: Manuel Weiss.)

Manuel Weiss is with the Compliant Robotics Lab, Berlin University of Applied Sciences and Technology, 10963 Berlin, Germany, and also with the Institute of Mechatronic Systems, Leibniz University Hannover, 30167 Hannover, Germany (e-mail: manuel.weiss@bht-berlin.de).

Alexander Pawluchin and Ivo Boblan are with the Compliant Robotics Lab, Berlin University of Applied Sciences and Technology, 10963 Berlin, Germany.

Jan-Hendrik Ewerling and Thomas Seel are with the Institute of Mechatronic Systems, Leibniz University Hannover, 30167 Hannover, Germany.

Digital Object Identifier 10.1109/LRA.2026.3653326

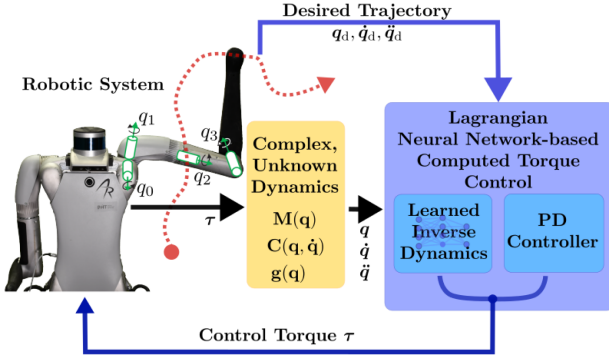


Fig. 1. Schematic of the proposed LNN-CTC framework. An LNN learns the inverse dynamics $M(q)$, $C(q, \dot{q})$, $g(q)$ to compute torque commands τ , combined with PD feedback. This enables accurate trajectory tracking ($q_d, \dot{q}_d, \ddot{q}_d$) without requiring an explicit analytic model.

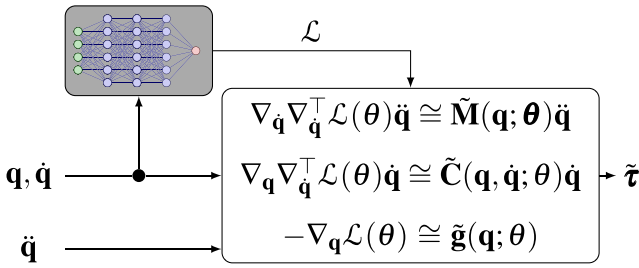


Fig. 2. Architecture of the LNN. The LNN takes as input the generalized coordinates q and generalized velocities $\dot{q}$. These inputs are processed by the LNN to produce a Lagrangian $\mathcal{L}$, which is differentiated to obtain the generalized forces or torques $\tilde{\tau}$ (5).

rather than adding a torque bias. When accurate, the LNN achieves decoupled linear error dynamics and improves rejection of disturbances. This changes the error dynamics and, therefore, the controller's stability, both of which are determined by the LNN, in contrast to feedforward control. Because LNN parameterize the Lagrangian energy of a system, it is possible to extract the exact terms required by CTC, making them a structure-preserving model replacement (Fig. 1).

This letter introduces LNN-CTC and demonstrates its effectiveness across multiple CTC architectures by comparing it with existing control strategies, including model-based CTC, LNN feedforward control and a proportional integral derivative (PID) control as a baseline. In particular, this letter proposes incorporating the LNN inverse-dynamics model directly into the CTC law (both full-model and feedback-linearization-only variants), so that when the learned model is accurate, the closed-loop reduces to decoupled linear error dynamics. With a bounded approximation error, the loop behaves as a linear system with an additive disturbance (standard CTC robustness). This tight integration is fundamentally different from existing LNN feedforward controllers, which do not alter the closed-loop linearization. Empirically, on a 4-degrees of freedom (DOF) arm, LNN-CTC trained with only 100 s of hardware data reduces the root mean square (RMS) error from 2.399° to 1.577° (-34.3 %) relative to model-based CTC with identical gains and across ten 130 s trajectories, and outperforms LNN feedforward and pure PD. These results indicate that placing a physics-structured

learner inside the feedback-linearized loop is both practically impactful and data-efficient.

The letter is structured as follows. In Section II, the proposed LNN-CTC approach is presented, including the considered class of robotic systems. The experimental setup is introduced in Section III-A. In Section III, the results are presented and a conclusion is drawn in Section IV.

II. METHODOLOGY

This section introduces the conventions used, preliminary work on LNN, and CTC.

A. Modeling the Robot's System Dynamics

Consider a robotic system with n -DOF. Let $q \in \mathbb{R}^n$ denote the vector of joint positions, $\dot{q} \in \mathbb{R}^n$ the vector of joint velocities, and $\ddot{q} \in \mathbb{R}^n$ the vector of joint accelerations. The state of the system is given by the pair $(q, \dot{q})$, and the control input is the vector of joint torques $\tau \in \mathbb{R}^n$ produced by the actuators. The dynamic behavior of the robot is described by the dynamic equations of motion

$$\tau(q, \dot{q}, \ddot{q}) = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q), \quad (1)$$

where $M(q) \in \mathbb{R}^{n \times n}$ is the mass-inertia matrix, $C(q, \dot{q}) \in \mathbb{R}^{n \times n}$ contains the centrifugal and Coriolis forces, and $g(q) \in \mathbb{R}^n$ are the gravitational forces. $\tau(q, \dot{q}, \ddot{q})$ denotes the actuator torque vector specified by the inverse dynamics for the current state and acceleration. $M(q)$, $C(q, \dot{q})$ and $g(q)$ in (1) are dependent on the robotic state configuration and need to be known or identified for diverse model-based control approaches. For instance, the mass matrix accounts for the inertial properties of the robot and varies with the configuration due to the mass distribution. The centrifugal and Coriolis matrix $C(q, \dot{q})$ includes terms that arise from the robot's motion and can significantly affect the dynamics at higher speeds. The gravitational vector $g(q)$ represents the torques required to counteract gravity in different configurations. To simplify the notation we combine Coriolis/centrifugal and gravitational terms, and define

$$f(q, \dot{q}) := C(q, \dot{q})\dot{q} + g(q). \quad (2)$$

B. Lagrangian Neural Networks

An LNN is a neural network designed to model physical systems by leveraging the principles of Lagrangian mechanics (Fig. 2). This method enables data efficiency and generalization beyond the training data distribution by restricting learning to the class of systems that can be modeled by Lagrangian mechanics, thus structuring and guiding the learning process. As a result, the obtained neural network respects the underlying physical principles, for example, the conservation of energy [11], [12], [15].

We define the Lagrangian

$$\mathcal{L}(q, \dot{q}) \equiv T(q, \dot{q}) - V(q, \dot{q}), \quad (3)$$

where T is the kinetic energy and V is the potential energy. It can be shown that the robot's inverse dynamics (1) can be expressed

in terms of the Lagrangian $\mathcal{L}$ as

$$\tau(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) = (\nabla_{\dot{\mathbf{q}}} \nabla_{\dot{\mathbf{q}}}^T \mathcal{L}(\mathbf{q}, \dot{\mathbf{q}})) \ddot{\mathbf{q}} + (\nabla_{\mathbf{q}} \nabla_{\dot{\mathbf{q}}}^T \mathcal{L}(\mathbf{q}, \dot{\mathbf{q}})) \dot{\mathbf{q}} - \nabla_{\mathbf{q}} \mathcal{L}(\mathbf{q}, \dot{\mathbf{q}}), \quad (4)$$

which relates changes in the system state to the control inputs [6]. The proposed control methodology addresses the robot's inverse dynamics, which can be reformulated as (1). Because $\mathbf{M}(\mathbf{q})$ and $\mathbf{f}(\mathbf{q}, \dot{\mathbf{q}})$ are traditionally calculated from measured masses, lengths, and moments of inertia, we instead express the inverse dynamics using an LNN as

$$\tilde{\tau}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}; \theta) = \tilde{\mathbf{M}}(\mathbf{q}; \theta) \ddot{\mathbf{q}} + \tilde{\mathbf{f}}(\mathbf{q}, \dot{\mathbf{q}}; \theta) \quad (5)$$

with $\tilde{\cdot}$ denoting approximations. θ makes explicit that $\tilde{\tau}$ depends on the parameters of the LNN (see Fig. 2).

Because the LNN learns the underlying equations of motion instead of a general state transition function, LNN need less data than traditional DNNs for good predictions [7], [11], [12], [16]. To obtain the parameters θ of the LNN, the discrepancy between the LNN's predictions with respect to the true torque is minimized:

$$\theta^* = \arg \min_{\theta} L(\tilde{\tau}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}; \theta), \tau_{\text{meas}}), \quad (6)$$

where L is a differentiable loss function, reflecting the mismatch between predicted torque and the measured torque (τ_{meas}) by the built-in sensors.

C. Controller

CTC is a widely used control technique for robotics and other nonlinear systems based on feedback linearization. Using an inverse robot model, the control task is transformed into a linearized problem, enabling the use of linear controllers [17], [18], [19], [20]. The fundamental concept of CTC can be implemented through various structural configurations that differ in their formulation and placement of model-based components within the control loop, but all incorporate a PID controller as an essential feedback element. The PID component allows the system to actively respond to tracking errors irrespective of the specific computed torque structure employed, ensuring robustness against model inaccuracies and external disturbances.

The PID input is the current state and the desired joint-space trajectory ($\mathbf{q}_d$, $\dot{\mathbf{q}}_d$, and $\ddot{\mathbf{q}}_d$) provided by a trajectory planner. The controller computes the error, its derivative, and its integral as follows

$$\begin{aligned} \mathbf{e}_P(t) &= \mathbf{q}_d(t) - \mathbf{q}(t), \\ \mathbf{e}_I(t) &= \int_0^t \mathbf{e}_P(t) dt, \\ \mathbf{e}_D(t) &= \dot{\mathbf{q}}_d(t) - \dot{\mathbf{q}}(t). \end{aligned} \quad (7)$$

The PID controller outputs either the feedback joint acceleration $\ddot{\mathbf{q}}_{\text{PID}}(t)$ or the control torque $\tau_{\text{PID}}(t)$, depending on the chosen controller setting, computed as

$$\mathbf{u}_{\text{PID}}(t) = \mathbf{K}_P \mathbf{e}_P(t) + \mathbf{K}_I \mathbf{e}_I(t) + \mathbf{K}_D \mathbf{e}_D(t), \quad (8)$$

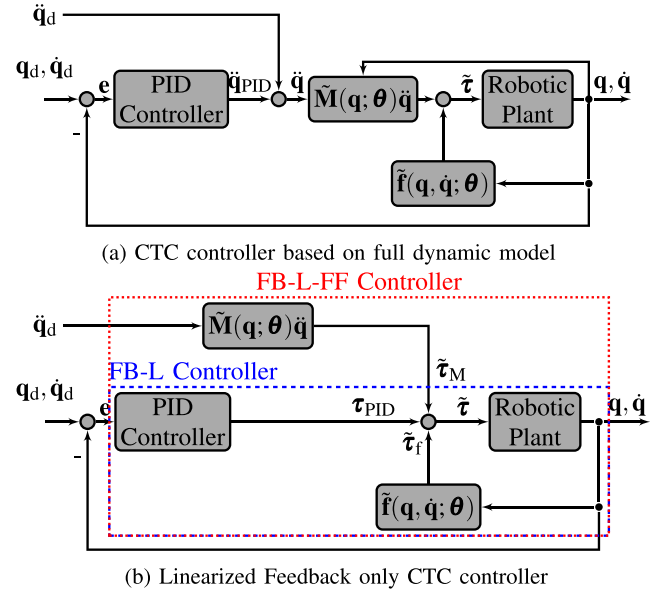


Fig. 3. Control loops for two common CTC architectures: (a) Full dynamic model CTC using PID control to compute desired joint accelerations $\ddot{\mathbf{q}}_d$. (b) Feedback-Linearization-based CTC (FB-L), which uses PID control to compute the required torque. The FB-L architecture can be enhanced by incorporating a feedforward term (FB-L-FF) based on the inertia matrix to add necessary torque for tracking $\ddot{\mathbf{q}}_d$.

where $\mathbf{u}_{\text{PID}}$ denotes the controller output ($\mathbf{u}_{\text{PID}} \equiv \ddot{\mathbf{q}}_{\text{PID}}$ or $\mathbf{u}_{\text{PID}} \equiv \tau_{\text{PID}}$), the diagonal matrices $\mathbf{K}_P, \mathbf{K}_I, \mathbf{K}_D \in \mathbb{R}^{n \times n}$ are the proportional, derivative, and integral gains, respectively.

1) *CTC*: CTC is a control approach for handling nonlinear dynamics in robotic systems that ensures accurate trajectory tracking by calculating the required torque to follow a desired trajectory through model inversion. This decouples the nonlinear dynamics of the system, resulting in a linearized model that is simpler to control. A standard feedback controller can then efficiently regulate the decoupled joint variables, leading to enhanced control precision [19]. In this work, two CTC architectures are considered: full-model (control variable $\ddot{\mathbf{q}}$) and feedback-linearization CTC (control variable τ).

2) *CTC With Full Dynamic Model*: The full dynamic model CTC consists of two components. The first component determines a compensation torque $\tilde{\tau}_f$ for (possibly nonlinear) effects in the system behavior, such as friction, Coriolis forces, and gravitational effects at the current operating point. It is solely dependent on the current state of the system (see Fig. 3(a)). The second component, $\tilde{\mathbf{M}}(\mathbf{q}; \theta) \ddot{\mathbf{q}}$, computes the torque required to achieve the desired joint acceleration at the current operating point. Therefore, it allows control in the joint space by providing the linear transformation between joint accelerations and torques. Here, $\ddot{\mathbf{q}}$ represents a sum of the feedforward acceleration $\ddot{\mathbf{q}}_{\text{ff}}$ and the feedback acceleration $\ddot{\mathbf{q}}_{\text{PID}}$, generated by the PID controller.

3) *CTC Feedback-Linearization*: The Feedback-Linearization-based CTC (FB-L) (see Fig. 3(b)) uses the torque $\tilde{\tau}_f$ as well. In contrast to the full CTC scheme, the architecture employs a PID torque control law

$$\tilde{\tau}(t) = \tau_{\text{PID}}(t) + \tilde{\tau}_f(t). \quad (9)$$

For improved performance, if the dynamic model is sufficiently accurate, the control law can be augmented with an additional torque term, $\tilde{\tau}_M = \tilde{M}(\mathbf{q}; \boldsymbol{\theta})\ddot{\mathbf{q}}_d$, which compensates for inertia during motion, resulting in a feedback-linearized CTC with acceleration feedforward (FB-L-FF). The resulting control law reads

$$\tilde{\tau}(t) = \tau_{\text{PID}}(t) + \tilde{\tau}_f(t) + \tilde{\tau}_M(t). \quad (10)$$

The proposed controller architectures can be found on GitHub (https://github.com/ma-weiss/Lagrangian-Neural-Net-work_CTC).

III. RESULTS

This section introduces experimental setups and the generation of training data. The results of the experiments are shown and discussed.

A. Experimental Setup and Training Data

All experiments are conducted on the 4-DOF arm of Agility Robotics' Digit platform (Fig. 1). The robot base is assumed to be stationary. Unless stated otherwise, the trained parameters of the LNN remain fixed. Only the adaptation study involves retraining. The evaluation comprises seven experiments: (1) Model generalization, comparing predicted torques for 250,000 out-of-distribution (OOD) states ($\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}$) against an analytic Lagrangian model, (2) Robustness, testing tracking under perturbed link parameters and uniformly distributed external wrenches on the end effector, (3) Gravity compensation, assessing static equilibrium to isolate the potential term V , (4) Dynamic trajectory tracking, evaluating acceleration-based LNN-CTC performance against diverse references with OOD setpoints, (5) Control-scheme comparison, evaluating torque-based LNN-CTC variants (FB-L, FB-L-FF) against different benchmarks, (6) Adaptation, adding a 3.5 kg payload ($\approx$ arm mass 3.6 kg) and fine-tuning the LNN to recover nominal tracking and (7) Cross-robot evaluation in simulation, extending the controller to the UR5 and KUKA IIWA 14 to study scalability. Training data are generated from smooth arbitrary trajectories tracked by a PD controller, with position setpoints sampled as $\mathbf{q} \sim \mathcal{U}(\mathbf{q}_{\min}, \mathbf{q}_{\max})$ and segment durations drawn from $\mathcal{U}(10\text{ s}, 25\text{ s})$. MuJoCo is used to generate simulated data, while analytic models and random poses rely on Pinocchio [21]. The desired trajectories are generated using the Python robotics toolbox [22]. Cross-source evaluation (simulation vs. hardware) further quantifies sim-to-real transfer and inductive bias.

For all experiments, we trained the LNN using 10 s, 100 s, 1000 s of simulated training data (Fig. 4), as well as LNN trained with 10 s and 100 s of real-world training data. All data were sampled at 500 Hz. This work uses the Mahalanobis norm as loss function because the magnitude of the residual might vary between different joints. The LNN is a fully connected MLP (4 hidden layers, 64 units each) with Glorot-uniform initialization.

On an Apple M1 Pro CPU, the LNN-CTC inference is approximately 0.2 ms (187.7 MFLOP/s at 500 Hz with 375,324 FLOPs per inference).

This work begins with open-loop adequacy and robustness evaluations, then reports closed-loop performance. The

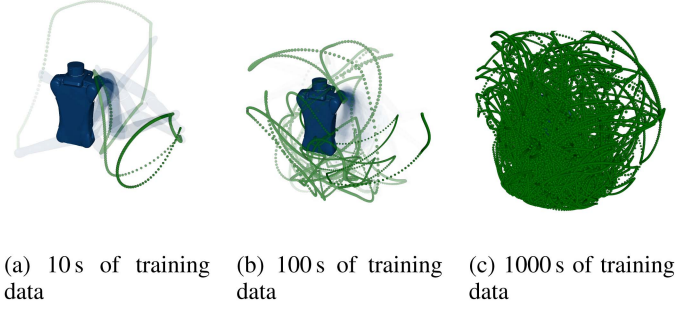


Fig. 4. Visualization of the trajectories over varying training data length. (a) With 10 s of training data, the sparse trajectories show a limited explored operational region. (b) With 100 s of training data, the trajectories are denser, showing more complex and varied movements with a greater explored operational region. (c) With 1000 s of training data, the behavior exhibits an extensively explored operational region.

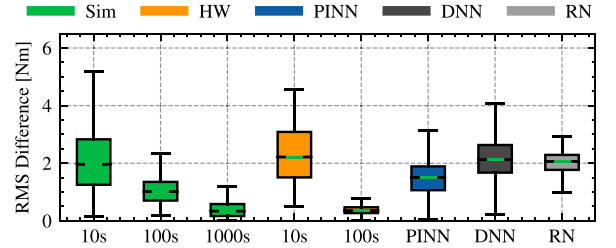


Fig. 5. RMS difference between the LNN-predicted torque and the torque estimated by the model, using 250,000 randomly sampled combinations of $\mathbf{q}$, $\dot{\mathbf{q}}$, and $\ddot{\mathbf{q}}$. Since the model-estimated $\tilde{\tau}$ is only approximate, it serves only as a baseline to measure how close the LNN's predictions are. The LNN trained on 1000 s simulated and 100 s hardware data most closely approximate the model. In contrast, the LNN trained on just 10 s of data exhibit larger offsets, with the simulated version having a greater error. DNN and RN are trained with 100 s hardware data. Bar color encodes the training data source for the inverse-dynamics model.

LNN-CTC achieves lower root mean square error (RMSE) than the model-based CTC with identical gains.

B. Open Loop Prediction

To evaluate the generalization capabilities of the LNN in predicting open-loop Lagrangian dynamics, the robot Lagrangian model is used as a baseline due to its reasonably good performance. Since the model-estimated $\tilde{\tau}$ is only approximate, it serves only as a baseline to measure how close the LNN's predictions are. To compare the LNN with the model, 250,000 arbitrary samples are drawn uniformly from the hyper-rectangle $[2\mathbf{x}_{\min}, 2\mathbf{x}_{\max}]$, where $\mathbf{x} = (\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$. These OOD data exceed hardware-feasible joint states ($\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}$). The RMS difference between the model-based Lagrangian torque and the LNN prediction (5) is computed for each data point (Fig. 5). As baselines, we include a PINN [23], a DNN to learn the full dynamics and a model-based Lagrangian with a DNN to learn the remaining error (RN) [24]. The baselines were trained with 100 s hardware data. The plot indicates that LNN predictions improve as more training data is used, especially when trained with hardware data. The lower RMS difference for models trained on 1000 s of simulated data and 100 s of hardware data suggests that the LNN become more accurate in predicting the required torque as the duration of training increases.

TABLE I
ORIGINAL SIMULATED ROBOT MODEL VS MODIFIED ROBOT MODEL VS.
MODIFIED ROBOT MODEL WITH EXTERNAL WRENCH ON THE END EFFECTOR
CONTROLLED BY LNN BASED ON 1000 s SIM. TRAINING DATA. FOR EACH
CONTROL ARCHITECTURE 100 NEW MODELS WERE SAMPLED.

	Original Model	Modified	External Wrench
CTC	$0.734^\circ \pm 0.059^\circ$	$1.741^\circ \pm 0.171^\circ$	$1.932^\circ \pm 0.172^\circ$
FB-L	$1.137^\circ \pm 0.078^\circ$	$2.680^\circ \pm 0.170^\circ$	$2.875^\circ \pm 0.164^\circ$
FB-L-FF	$1.046^\circ \pm 0.090^\circ$	$2.929^\circ \pm 0.165^\circ$	$3.170^\circ \pm 0.167^\circ$
FF [11]	$1.241^\circ \pm 0.086^\circ$	$3.139^\circ \pm 0.192^\circ$	$3.834^\circ \pm 0.199^\circ$

In contrast, LNN trained on limited data (10 s Sim and 10 s HW) exhibit significant RMS differences, implying reduced extrapolation to unseen joint-state combinations. The model trained on 10 s of simulated data shows the highest error, indicating that the quantity and type of training data significantly affect the accuracy and generalization of the LNN. The baselines show that DNN require more data than LNN to achieve similar performance. The baselines perform significantly worse than the LNN trained on the same data.

C. Robustness Evaluation

We evaluate robustness to structured uncertainty in simulation by sampling 100 modified robot models from bounded, physically motivated intervals derived from CAD tolerances and identification residuals (link length $\pm 10\%$, masses $\pm 15\%$, inertias $\pm 25\%$, center of mass (COM) ± 15 mm, friction $\pm 50\%$). Since parameter modification of the physical robot is not feasible, this is done in simulation. For an additional experiment, we added a uniformly distributed external wrench to the modified model (sampled as $\mathbf{w} \sim \mathcal{U}(-20, 20)^6$) to validate the robustness of the control architecture. We employ Latin Hypercube sampling to cover the space. The 95th percentile RMSE remained $< 3.253^\circ$ for all architectures. Although we report only the LNN trained with 1000 s simulation data here, we hypothesize similar trends for all other LNN. This remains to be validated. The performance of the LNN-CTC degrades as approximation error increases. A similar effect is visible in the sim-to-real transfer. However, LNN-CTC can handle imperfections in the model better than an LNN used as an FF controller (Table I). The performance of FB-L and FB-L-FF indicates that imperfect $\tilde{\mathbf{M}}(\mathbf{q}; \boldsymbol{\theta})$ has a substantial impact on the control performance. The results show that the FF controller has more difficulties in handling disturbances.

Preliminary sensitivity analysis indicates that mass and link length are the most influential. To reduce the performance below that of the PD controller ($3.912^\circ \pm 0.199^\circ$) the mass and the link length must be changed by $\approx 25\%$. However, this might be due to the low mass and therefore low torques of the LNN and might differ for other robots. We did not evaluate the robustness to disturbance (impulses, latency), which we leave for future work.

D. Gravity Compensation

The third experiment is conducted to test the ability to compensate for gravitational forces on the robot hardware. In particular, the term $\tilde{\mathbf{g}}(\mathbf{q}; \boldsymbol{\theta})$ should counteract gravitational forces, and

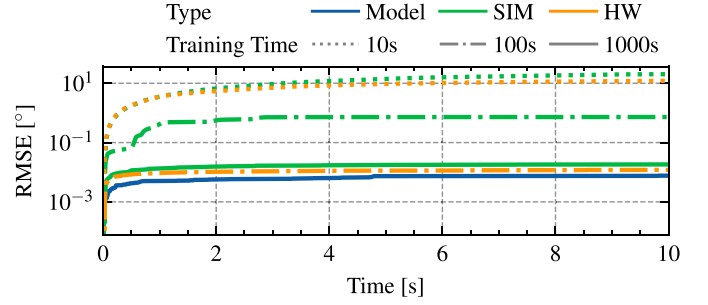


Fig. 6. Gravity compensation performance on hardware using different amounts of LNN training data. Simulation (SIM) and hardware (HW) training data are compared. While 10 s of training data is insufficient for equilibrium, both 100 s of hardware data and 1000 s of simulation data achieve satisfactory gravity compensation.

we test the approximation based on different amounts of training data. For this, the robot arm is placed in arbitrary positions using a PD controller. These positions should be maintained without feedback if the model is sufficiently good (see Fig. 6). This is only done for LNNs and the analytic model, since isolating the gravity term from DNN is not feasible without effectively implementing a CTC variant.

As a reference, the experiment is also conducted using the original model-based controller. Using only 10 s of simulated data to train the LNN, the controller is not able to keep the robotic arm in equilibrium, and the arm always yields to gravity, resulting in the arm hanging down. Only the elbow joint maintains its position due to its low weight and high friction. With 10 s of training data collected on real hardware, the results are better than with simulated training data. Still, the controller can only achieve equilibrium for the robotic arm close to hanging down. Therefore, within 10 s the exploration of the joint space configuration is not sufficient to extrapolate to configurations not included in the training data. Using 100 s or more data to train the LNN results in an error $< 1^\circ$ (see Fig. 6 and Table II). Training data gathered on the real hardware outperform the simulated training data due to the implicit learning of some friction components that are not simulated using the manufacturer's MuJoCo model due to fixed joint damping and friction components in the simulation. As expected, none of the LNN-based gravity models $\tilde{\mathbf{g}}(\mathbf{q}; \boldsymbol{\theta})$ can outperform the model-based approach, since the masses are perfectly known in the model.

E. Reference Tracking CTC

To validate the acceleration-based CTCs' performance on different amounts of training data, controllers were tested on 10 arbitrary trajectories of 130 s each. The trajectories vary in speed and amplitude to cover the robot's operational space, deliberately adding partially OOD conditions (higher velocities, with segment durations drawn from $\mathcal{U}(2\text{ s}, 35\text{ s})$), diverse motions, and joint configurations not present in the training set. The integral term is omitted from the PID controller, creating a PD controller. All parameters remain fixed throughout the experiments to ensure an unbiased comparison, with model-based, other neural network-based CTC as the baselines. As additional baselines a PINN, a DNN and an RN were trained with 100 s of

TABLE II

RMSE AND STANDARD DEVIATION (STD) IN DEGREES OVER 10 TRAJECTORIES OF 130 s OF HARDWARE EXPERIMENTS. LNN MODELS WERE TRAINED ON VARYING AMOUNTS OF SIMULATED (SIM.) OR EXPERIMENTAL (EXP.) DATA. RMSE IS THE MEAN ACROSS THE 10×130 s TRAJECTORIES. THE RED AND GREEN LINES ARE BARPLOTS OF THE RMSE INDICATING THE BEST PERFORMING IN GREEN. AS AN ADDITIONAL BASELINE A PD CONTROLLER ALONE ACHIEVED A RMSE OF 3.454° . (N.A.: NOT APPLICABLE, UNST.: UNSTABLE CLOSED-LOOP TRACKING).

		Gravity Comp.	CTC	FB-L	FB-L-FF	FF [11]
LNN based on sim. training data						
10 s	RMSE $\pm$ STD	$14.676^\circ \pm 12.687^\circ$	$14.362^\circ \pm 0.464^\circ$	$3.238^\circ \pm 0.266^\circ$	$3.525^\circ \pm 0.324^\circ$	$3.552^\circ \pm 0.333^\circ$
100 s	RMSE $\pm$ STD	$0.688^\circ \pm 0.595^\circ$	$1.601^\circ \pm 0.330^\circ$	$2.602^\circ \pm 0.203^\circ$	$2.812^\circ \pm 0.186^\circ$	$2.770^\circ \pm 0.208^\circ$
1000 s	RMSE $\pm$ STD	$0.017^\circ \pm 0.01^\circ$	$1.567^\circ \pm 0.379^\circ$	$2.146^\circ \pm 0.190^\circ$	$2.016^\circ \pm 0.365^\circ$	$2.357^\circ \pm 0.195^\circ$
LNN based on exp. training data						
10 s	RMSE $\pm$ STD	$1.962^\circ \pm 1.53^\circ$	$1.931^\circ \pm 0.344^\circ$	$3.348^\circ \pm 0.183^\circ$	$3.495^\circ \pm 0.175^\circ$	$3.482^\circ \pm 0.203^\circ$
100 s	RMSE $\pm$ STD	$0.011^\circ \pm 0.003^\circ$	$1.577^\circ \pm 0.317^\circ$	$2.076^\circ \pm 0.190^\circ$	$1.831^\circ \pm 0.338^\circ$	$2.085^\circ \pm 0.193^\circ$
Baselines / others						
Model	RMSE $\pm$ STD	$0.011^\circ \pm 0.003^\circ$	$2.399^\circ \pm 0.530^\circ$	$2.076^\circ \pm 0.191^\circ$	$2.224^\circ \pm 0.162^\circ$	$2.133^\circ \pm 0.195^\circ$
PINN [23]	RMSE $\pm$ STD	n.a.	$2.361^\circ \pm 0.214^\circ$	$2.520^\circ \pm 0.668^\circ$	$2.359^\circ \pm 0.214^\circ$	$2.359^\circ \pm 0.214^\circ$
DNN	RMSE $\pm$ STD	n.a.	unst.	$3.333^\circ \pm 0.416^\circ$	$3.026^\circ \pm 0.336^\circ$	$3.799^\circ \pm 0.234^\circ$
RN [24]	RMSE $\pm$ STD	n.a.	unst.	$2.452^\circ \pm 0.267^\circ$	$2.594^\circ \pm 0.309^\circ$	$2.565^\circ \pm 0.225^\circ$

hardware data. The DNN-based baselines were unstable due to poor extrapolation and to their placement within the feedback loop.

The results in Table II reveal a clear hierarchy in training data effectiveness. LNN-CTC trained on only 10 s of simulated data failed to produce meaningful torques, resulting in poor trajectory tracking performance. However, 10 s of hardware training data achieved a performance approaching the model-based baseline, highlighting the superior representational quality of the real-world data. LNN-CTC trained on 100 s of hardware data outperformed the model-based CTC baseline. This superior performance could be attributed to the implicit learning of unmodeled dynamics, particularly friction and damping terms that are challenging to model analytically but naturally captured in hardware data. The LNN's ability to learn these real-world effects compensates for the limitations of the manufacturer's simulation model.

Fig. 7 compares the tracking performance between model-based CTC and the best-performing LNN-CTCs (1000 s simulated and 100 s hardware data). For clarity, only the analytical model is plotted as a baseline. The PINN-CTC performs only marginally better than the model-base and remains far below the LNN-based CTC (Table II), so including it would not change the qualitative comparison. The LNN-based controllers demonstrate consistently lower tracking error across the majority of the trajectory, validating their practical advantage over traditional model-based approaches. The advantage is most evident in high-acceleration segments, where model-based error spikes are markedly reduced.

The results show that LNN-CTC achieve superior performance with minimal training data requirements when using hardware data, while maintaining the interpretability and physical consistency inherent to Lagrangian mechanics. This combination of data efficiency and performance improvement makes

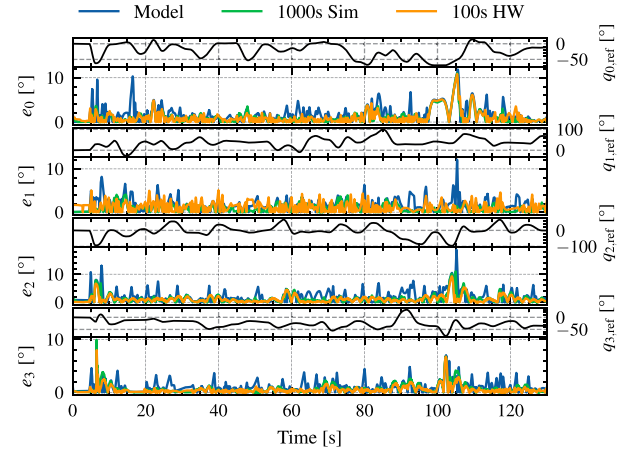


Fig. 7. Tracking performance of model-based CTC with LNN-CTCs using different training data (1000 s simulated data (SIM) and 100 s data gathered on the hardware (HW)).

LNN-CTC particularly suitable for real-world robots where accurate system models are difficult to obtain.

F. Reference Tracking Feedback-Linearization

To isolate the performance contributions of different control components, we compare the two torque-based LNN-CTC architectures (FB-L and FB-L-FF) with the LNN-based feed-forward controller (FF) using identical control parameters. All controllers use unified PD gains to ensure fair comparison: proportional gain $\mathbf{K}_P = 50 \cdot \mathbf{I}_4$ and derivative gain $\mathbf{K}_D = 5 \cdot \mathbf{I}_4$ where $\mathbf{I}_4$ denotes the identity matrix of order 4. The integral term is omitted, creating a PD controller configuration. Each controller tracks the same trajectory set used in previous experiments, with all LNN trained on 10 s of hardware data. The PD

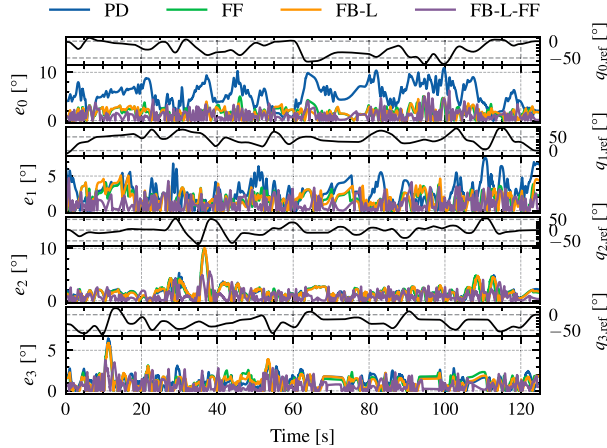


Fig. 8. Comparing a PD controller with various LNN-based control architectures, all trained on 100 s of hardware data and using identical PD gains. The PD controller performs the worst, while the feedforward controller (FF) [11] and the feedback-linearized controller (FB-L) show similar performance. The feedback-linearized controller performs best with $\ddot{q}_d$ feedforward (FB-L-FF).

controller alone achieves a RMSE of 3.454° , demonstrating limited performance and highlighting the necessity of model-based compensation for accurate trajectory tracking. As additional baselines, we include a PINN, a DNN and an RN.

Fig. 8 shows the reference trajectories and tracking errors for the PD, FF, FB-L, and FB-L-FF controllers. PD exhibits the largest and most variable errors, while FF and FB-L produce similar, substantially tighter tracking. FB-L-FF achieves the smallest and most consistent errors across all joints. The superior performance of FB-L-FF demonstrates that the incorporation of the learned inertia matrix $\tilde{M}(q; \theta)$ for acceleration feedforward provides significant benefits. The learned inertia model is more accurate than its model-based counterpart, enabling a better torque computation for desired accelerations. The DNN and the RN can track the trajectories but are less accurate than the LNN with the same training data. The 10 s simulation trained LNN outperforms the PD controller in these architectures, which mainly contributed to the poor fitted $\tilde{M}(q; \theta)$ not being inside the feedback loop.

The results confirm that integrating learned dynamics into feedback-linearization and feedforward control architectures significantly enhances precision and robustness compared to simpler control approaches. The FB-L-FF architecture effectively combines the benefits of disturbance rejection (through feedback) and accurate dynamic compensation (through learned feedforward terms).

G. Payload Adaptation

To assess adaptability, a payload of 3.5 kg, nearly equal to the arm's own mass 3.6 kg, is attached to the end-effector, causing a strong change in the inertial and gravitational dynamics of the system. The pre-trained LNN used in the previous experiments is fine-tuned on the data collected under this new configuration. In ten trials, the controller initially failed to produce meaningful torques and collapsed, but recovered stable tracking within 10 s of additional data. Fig. 9 shows one of the trials. The adapted

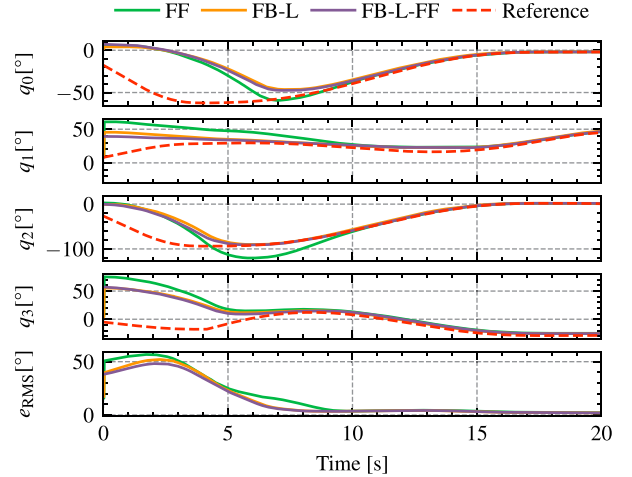


Fig. 9. With a 3.5 kg payload, nearly equal to the arm's mass, the LNN-CTC regained nominal tracking within 10 s of additional data, reducing the RMS joint-space error from over 40° to below 5° and demonstrating rapid, data-efficient adaptation to large dynamic changes.

TABLE III
MEAN JOINT-SPACE RMSE $\pm$ STD OF OVER 10×130 s TRAJECTORIES FOR THE FB-L-FF CONTROLLER ON THREE ROBOT PLATFORMS IN MUJoCo SIMULATION. RESULTS SHOW THAT COARSE TREND TRACKING IS ACHIEVED WITH LITTLE DATA, WHILE HIGHER DOF SYSTEMS REQUIRE PROPORTIONALLY MORE DATA TO REACH COMPARABLE ACCURACY.

	Digit Arm (4-DOF)	UR5 (6-DOF)	KUKA IIWA 14 (7-DOF)
10 s	$2.901^\circ \pm 1.309^\circ$	$3.821^\circ \pm 1.671^\circ$	$4.621^\circ \pm 1.972^\circ$
100 s	$2.071^\circ \pm 0.108^\circ$	$2.890^\circ \pm 0.087^\circ$	$3.015^\circ \pm 0.094^\circ$
1000 s	$1.146^\circ \pm 0.090^\circ$	$1.541^\circ \pm 0.065^\circ$	$1.730^\circ \pm 0.107^\circ$

LNN-CTC reduced the RMSE from above 40° to below 5° , restoring smooth torque profiles without manual reparameterization. These results confirm the capacity of the LNN for fast and data-efficient adaptation to large structural changes, an essential property for practical manipulation tasks.

H. Additional Robots

To examine generalization across morphologies, the feedback-linearized LNN with feedforward compensation (FB-L-FF) was evaluated on two additional robot platforms in simulation of increasing complexity: The 4-DOF Digit arm, the 6-DOF UR5 and the 7-DOF KUKA IIWA 14 with models from the MuJoCo Menagerie. As shown in Table III, all systems achieved stable tracking with limited data, and performance improved consistently with longer training durations. After only 10 s of training data, the joint-space average RMSE remained below 5° across all robots, demonstrating that coarse trend tracking can be achieved with minimal data. With 1000 s of data, the Digit arm reached 1.15° , UR5 1.54° , and KUKA IIWA 14 1.73° mean RMSE over 10 trajectories of 130 s with a similar magnitude of the desired joint velocities and joint accelerations, indicating a clear scaling relationship between model complexity, data requirements and operational space dimensionality. These results confirm that while the LNN-based controller generalizes across platforms, higher DOF systems demand proportionally more data to achieve comparable accuracy, reflecting the inherent

increase in dynamic coupling and task complexity. Cross-robot results demonstrate sim-to-sim generalization across morphologies, but sim-to-real transfer on other platforms remains future work.

Overall, these results demonstrate that LNN-based CTC require sufficient training data for stable performance, with hardware-collected data consistently outperforming simulation data of equivalent duration. The FB-L-FF architecture proved to be most effective by combining learned dynamics compensation with acceleration feedforward. For 10 s of simulated training data, gravity compensation cannot maintain equilibrium and LNN-CTC tracking shows a large error (RMSE 14.3°) but small changes in architecture (FB-L/FB-L-FF) achieve better performance than the PD controller ($\approx 3.2^\circ$ to 3.5° RMSE vs. 3.454°). In the settings and platforms evaluated, an interaction window of 10 s proved to be insufficient. Achieving robustness requires interaction on the order of 100 s.

IV. CONCLUSION

This letter introduces and evaluates LNN for linearized feedback control, specifically focusing on CTC. By accurately capturing real-world system behaviors from limited training data and extrapolating to OOD regimes, LNN-CTC significantly enhances control performance.

Experimental validation on a 4-DOF humanoid robotic arm confirms that LNN-CTC achieves superior trajectory tracking precision compared to conventional model-based CTC. Remarkably, this was accomplished using only 100 s of hardware-collected training data. By inherently compensating for unmodeled dynamics, LNN-based controllers outperform their model-based counterparts, demonstrating tracking accuracy improvements exceeding 30 % in certain cases. The results underline the potential of physics-informed neural architectures to offer robust, interpretable, and highly data-efficient control solutions. Such capabilities are particularly beneficial in practical robotics applications, where obtaining complete analytical system models is often infeasible due to system complexity and cost constraints.

To our knowledge, this is the first integration of LNN into linearized feedback control schemes such as CTC. By unifying physics-informed modeling and real-time feedback control, LNN-CTC combines interpretability, robustness, and plug-and-play scalability.

Future work will apply this strategy to more complex systems and examine its scalability in dynamic robotic environments.

REFERENCES

- [1] S. Nahavandi, R. Alizadehsani, D. Nahavandi, C. P. Lim, K. Kelly, and F. Bello, "Machine learning meets advanced robotic manipulation," *Inf. Fusion*, vol. 105, 2024, Art. no. 102221.
- [2] Z. Zhang and Z. Chen, "Modeling and control of robotic manipulators based on symbolic regression," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 5, pp. 2440–2450, May 2023.
- [3] T. N. Truong, A. T. Vo, and H.-J. Kang, "A model-free terminal sliding mode control for robots: Achieving fixed-time prescribed performance and convergence," *ISA Trans.*, vol. 144, pp. 330–341, 2024.
- [4] J. Ibarz, J. Tan, C. Finn, M. Kalakrishnan, P. Pastor, and S. Levine, "How to train your robot with deep reinforcement learning: Lessons we have learned," *Int. J. Robot. Res.*, vol. 40, no. 4/5, pp. 698–721, 2021.
- [5] V. Mai, K. Mani, and L. Paull, "Sample efficient deep reinforcement learning via uncertainty estimation," in *Proc. Int. Conf. Learn. Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=vrW3tvDfOJQ>
- [6] A. R. Geist and S. Trimpe, "Structured learning of rigid-body dynamics: A survey and unified view from a robotics perspective," *GAMM-Mitteilungen*, vol. 44, no. 2, 2021, Art. no. e202100009.
- [7] M. Lutter and J. Peters, "Combining physics and deep learning to learn continuous-time dynamics models," *Int. J. Robot. Res.*, vol. 42, no. 3, pp. 83–107, 2023.
- [8] M. Meindl, S. Bachhuber, and T. Seel, "AI-Mole: Autonomous iterative motion learning for unknown nonlinear dynamics with extensive experimental validation," *Control Eng. Pract.*, vol. 145, 2024, Art. no. 105879.
- [9] J. Nicodemus, J. Kneifl, J. Fehr, and B. Unger, "Physics-informed neural networks-based model predictive control for multi-link manipulators," *IFAC-PapersOnLine*, vol. 55, no. 20, pp. 331–336, 2022.
- [10] M. Bensch, T.-D. Job, T.-L. Habich, T. Seel, and M. Schappler, "Physics-informed neural networks for continuum robots: Towards fast approximation of static cosserat rod theory," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2024, pp. 17293–17299.
- [11] M. Lutter, C. Ritter, and J. Peters, "Deep Lagrangian networks: Using physics as model prior for deep learning," in *Proc. Int. Conf. Learn. Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=BklHjCqKkm>
- [12] M. D. Cranmer, S. Greydanus, S. Hoyer, P. W. Battaglia, D. N. Spergel, and S. Ho, "Lagrangian neural networks," in *Proc. Workshop Integration Deep Neural Models Differ. Equ.*, 2019. [Online]. Available: <https://openreview.net/forum?id=iE8tFa4Nq>
- [13] J. K. Gupta, K. Menda, Z. Manchester, and M. Kochenderfer, "Structured mechanical models for robot learning and control," in *Proc. 2nd Conf. Learn. Dyn. Control*, 2020, pp. 328–337.
- [14] L. Schulze, J. Peters, and O. Arenz, "Context-aware deep lagrangian networks for model predictive control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2025.
- [15] H. Sharma, D. A. Najera-Flores, M. D. Todd, and B. Kramer, "Lagrangian operator inference enhanced with structure-preserving machine learning for nonintrusive model reduction of mechanical systems," *Comput. Methods Appl. Mech. Eng.*, vol. 423, 2024, Art. no. 116865.
- [16] M. Lutter, K. Listmann, and J. Peters, "Deep lagrangian networks for end-to-end learning of energy-based control for under-actuated systems," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 7718–7725.
- [17] S. Han, H. Wang, Y. Tian, and N. Christov, "Time-delay estimation based computed torque control with robust adaptive RBF neural network compensator for a rehabilitation exoskeleton," *ISA Trans.*, vol. 97, pp. 171–181, 2020.
- [18] D. Jorge, G. Pizzuto, and M. Mistry, "Efficient learning of inverse dynamics models for adaptive computed torque control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 11203–11 208.
- [19] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*, 1st ed. New York, NY, USA: Cambridge Univ. Press, 2017.
- [20] P. Masarati, "Computed torque control of redundant manipulators using general-purpose software in real-time," *Multibody Syst. Dyn.*, vol. 32, pp. 403–428, 2014.
- [21] J. Carpentier et al., "The Pinocchio C library—A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives," in *Proc. IEEE Int. Symp. Syst. Integrations*, 2019, pp. 614–619.
- [22] P. Corke and J. Haviland, "Not your grandmother's toolbox—The robotics toolbox reinvented for python," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 11357–11 363.
- [23] A. Al-Shahrabi, M. J. Javid, A. A. Fahmy, C. A. Griffiths, and C. Li, "Torque tracking position control of DLR-Hit II robotic hand using a real-time physics-informed neural network," *Appl. Math. Modelling*, vol. 145, 2025, Art. no. 116110.
- [24] J. Gruenstein, T. Chen, N. Doshi, and P. Agrawal, "Residual model learning for microrobot control," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 7219–7226.